Práctica 5: Construyendo un Agente con OpenRouter y Stitch MCP

Oct 20, 2026 · Britney Mishel Félix López

Datos generales

Campo	Dato
Alumna	Britney Mishel Félix López
Carrera	Ingeniería en Entornos Virtuales y Negocios Digitales
Universidad	Universidad Tecnológica Metropolitana
Cuatrimestre y grupo	7.º cuatrimestre, grupo 7-E
Práctica	Construyendo un Agente con OpenRouter y Stitch MCP

Objetivo de la práctica

Construir un agente autónomo de Inteligencia Artificial capaz de razonar, tomar decisiones y ejecutar herramientas en la máquina local para diseñar interfaces de usuario. **OpenRouter** funciona como el "cerebro" (LLM) y **Stitch MCP** como las "manos" (herramientas).

Requisitos previos

Requisito	Cómo lo cumplí
Cuenta en OpenRouter con saldo o créditos y una API Key	Cuenta de OpenRouter con API Key; modelo principal Claude Haiku 4.5 y modelos gratuitos de respaldo
Acceso a Stitch	API Key de Stitch (stitch.withgoogle.com → Settings → API Keys)
Node.js	Node.js con npm en Windows

Paso 1: Preparar el entorno de trabajo

Creé la carpeta del proyecto, inicialicé Node.js e instalé las dependencias:

```
mkdir agente-ui-stitch
cd agente-ui-stitch
npm init -y
npm install openai @modelcontextprotocol/sdk dotenv @google/stitch-sdk
```

En `package.json` agregué `"type": "module"` para usar módulos ES6:

```
{
  "name": "agente-ui-stitch",
  "type": "module",
  "main": "agente.js",
  "dependencies": {
    "@google/stitch-sdk": "^0.3.5",
    "@modelcontextprotocol/sdk": "^1.23.0",
    "dotenv": "^16.4.5",
    "openai": "^4.77.0"
  }
}
```

Además de las tres dependencias del manual instalé `@google/stitch-sdk`, que es el paquete oficial de Google que contiene el servidor MCP de Stitch.

Paso 2: Configurar las credenciales

Creé el archivo `.env` en la raíz del proyecto con las claves (el archivo no se comparte ni se sube a Git, por eso está en `.gitignore`):

```
OPENROUTER_API_KEY=tu_clave_de_openrouter_aqui
STITCH_API_KEY=tu_clave_de_stitch_aqui
OPENROUTER_MODEL=anthropic/claude-haiku-4.5
```

`STITCH_API_KEY` la necesita el servidor MCP de Stitch para autenticarse, y `OPENROUTER_MODEL` permite cambiar de modelo sin tocar el código.

Paso 3: Escribir el cerebro del agente

SYSTEM PROMPT

> *Eres un Ingeniero Frontend y Experto en UX/UI. Tu objetivo es traducir los requerimientos del usuario en interfaces funcionales usando las herramientas de Stitch. Reglas: 1. Analiza el requerimiento y decide qué herramientas usar. 2. Si una herramienta falla, lee el error, ajusta los parámetros y vuelve a intentarlo. 3. Informa al usuario de tus acciones de forma concisa.*

USER PROMPT

> *Crea un proyecto nuevo y genera una pantalla de inicio de sesión con un botón azul oscuro, la cual reciba un usuario y password, así como las opciones de registrarse, olvidé mi contraseña y recordar sesión siempre. Detalles adicionales: diseño para escritorio, fondo claro, tarjeta centrada con el título "Iniciar sesión", campos con etiqueta e icono, el checkbox "Recordar sesión siempre", el enlace "¿Olvidaste tu contraseña?" y abajo "¿No tienes cuenta? Regístrate". Todos los textos en español.*

La primera parte es el prompt del manual; los detalles adicionales los agregué como permite la nota de la práctica.

Código del agente (`agente.js`)

Partí del código del manual y completé lo que le faltaba para ejecutarse: cargar el `.env`, crear el cliente de OpenRouter y declarar los mensajes y la variable del bucle.

Cliente de OpenRouter (usa el SDK de OpenAI porque OpenRouter es compatible):

```
dotenv.config();

const openai = new OpenAI({
  baseURL: "https://openrouter.ai/api/v1",
  apiKey: process.env.OPENROUTER_API_KEY,
});

const mensajes = [
  { role: "system", content: SYSTEM_PROMPT },
  { role: "user", content: USER_PROMPT },
];
let tareaCompletada = false;
```

1. Levantar el servidor local de Stitch (MCP por stdio):

```
const transport = new StdioClientTransport({
  command: process.execPath,      // node
  args: ["stitch-mcp.js"],
  env: { ...process.env, STITCH_API_KEY },
});
const mcpClient = new Client({ name: "agente-stitch", version: "1.0" },
{ capabilities: {} });
await mcpClient.connect(transport);
```

El servidor MCP local es el archivo `stitch-mcp.js`, que usa el paquete oficial de Stitch:

```
import { StitchProxy } from "@google/stitch-sdk";
import { StdioServerTransport } from
"@modelcontextprotocol/sdk/server/stdio.js";

const proxy = new StitchProxy({ apiKey: process.env.STITCH_API_KEY });
await proxy.start(new StdioServerTransport());
```

2. Traducir las herramientas de MCP al formato de OpenRouter:

```
const mcpTools = await mcpClient.listTools();
const openRouterTools = mcpTools.tools.map(tool => ({
  type: "function",
  function: { name: tool.name, description: tool.description, parameters:
tool.inputSchema },
}));
```

3. El bucle del agente (Agentic Loop): el modelo decide qué herramienta usar, el agente la ejecuta con `mcpClient.callTool`, devuelve el resultado (o el error) a la memoria del modelo con el rol `tool`, y repite hasta que el modelo responde solo con texto:

```
while (!tareaCompletada) {
  const respuesta = await openai.chat.completions.create({
    model: MODELO,
    messages: mensajes,
    tools: openRouterTools,
    max_tokens: 2000,
```

```
  });
  const mensajeModelo = respuesta.choices[0].message;
  mensajes.push(mensajeModelo);

  if (mensajeModelo.tool_calls?.length > 0) {
    for (const toolCall of mensajeModelo.tool_calls) {
      try {
        const argumentos = JSON.parse(toolCall.function.arguments);
        const resultado = await mcpClient.callTool(
          { name: toolCall.function.name, arguments: argumentos },
          undefined,
          { timeout: 10 * 60e3 }   // generar una pantalla tarda varios
minutos
        );
        mensajes.push({ role: "tool", tool_call_id: toolCall.id,
                        name: toolCall.function.name, content:
JSON.stringify(resultado) });
      } catch (error) {
        mensajes.push({ role: "tool", tool_call_id: toolCall.id,
                        name: toolCall.function.name, content: `Error: $
{error.message}` });
      }
    }
  } else {
    console.log(`Agente: ${mensajeModelo.content}`);
    tareaCompletada = true;
  }
}
```

También agregué un límite de 8 vueltas para no gastar créditos de más, y que todo lo que sale en consola se guarde en `salida_agente.txt` como evidencia.

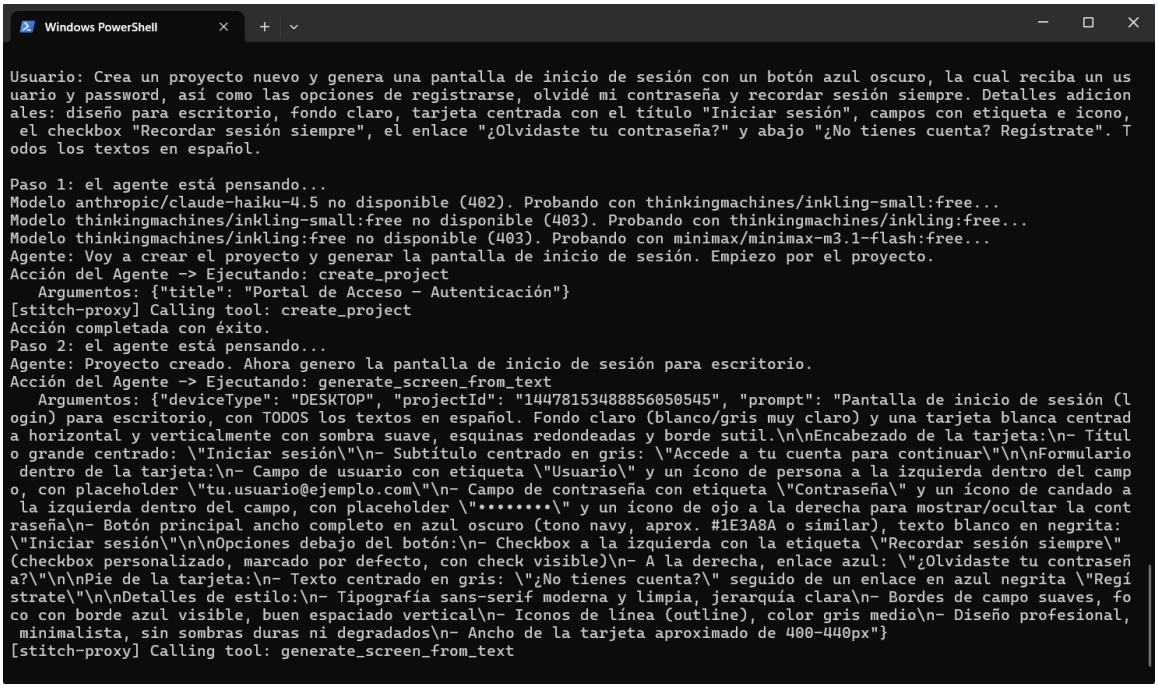
Para que el agente siempre termine, la llamada al modelo pasa por la función `pedirRespuesta()`: si el modelo principal responde con error de saldo (402) o de disponibilidad (403/404), el agente prueba automáticamente con el siguiente modelo gratuito de OpenRouter que soporte herramientas. Y si llega al límite de pasos, se le pide que deje de usar herramientas y entregue su resumen final.

Paso 4: Ejecución y pruebas

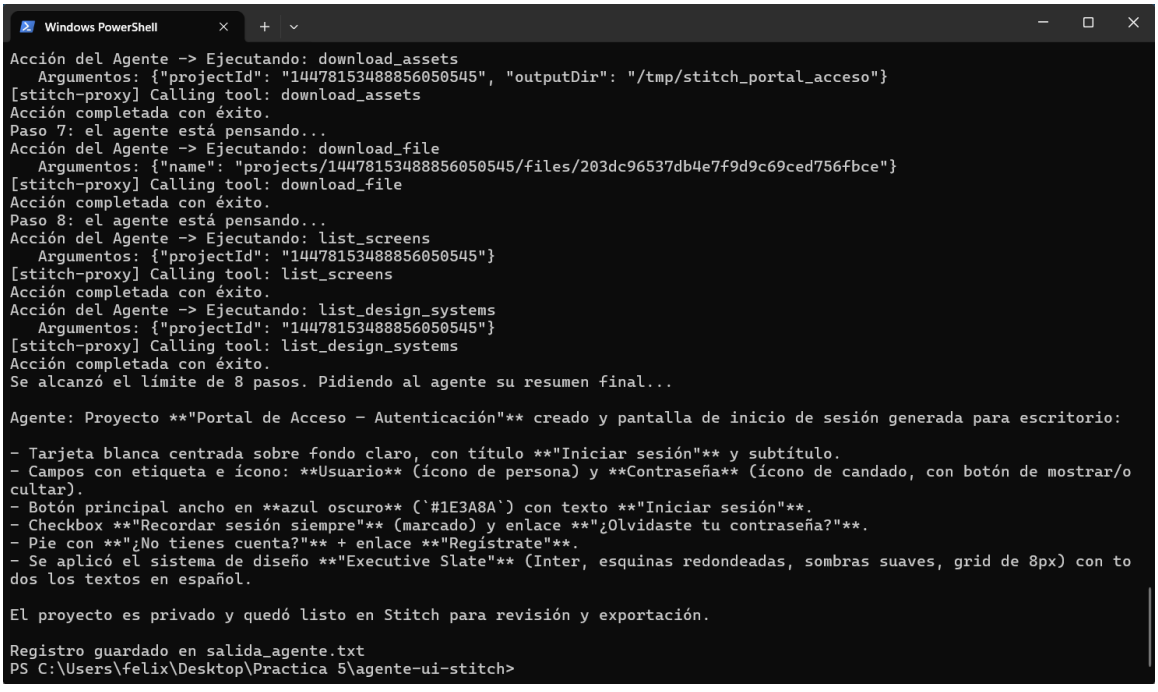
Abrí PowerShell en la carpeta del proyecto y ejecuté el script:

```
node agente.js
```

Ejecución en la terminal



Captura 1. Inicio de la ejecución: el agente cambia al modelo de respaldo, crea el proyecto con `create_project` y manda a generar la pantalla con `generate_screen_from_text`.



Captura 2. Final de la ejecución: el agente revisa su resultado con otras herramientas de Stitch y entrega el resumen final.

Registro completo de la consola (guardado en `salida_agente.txt`; el prompt largo que redactó el agente se muestra recortado):

```
Modelo: anthropic/claude-haiku-4.5

Iniciando conexión con Stitch MCP...
Conexión exitosa. 17 herramientas disponibles.

Paso 1: el agente está pensando...
Modelo anthropic/claude-haiku-4.5 no disponible (402). Probando con
thinkingmachines/inkling-small:free...
Modelo thinkingmachines/inkling-small:free no disponible (403). Probando con
thinkingmachines/inkling:free...
Modelo thinkingmachines/inkling:free no disponible (403). Probando con
minimax/minimax-m3.1-flash:free...
Agente: Voy a crear el proyecto y generar la pantalla de inicio de sesión.
Empiezo por el proyecto.
Acción del Agente -> Ejecutando: create_project
   Argumentos: {"title": "Portal de Acceso — Autenticación"}
Acción completada con éxito.
Paso 2: el agente está pensando...
Agente: Proyecto creado. Ahora genero la pantalla de inicio de sesión para
escritorio.
Acción del Agente -> Ejecutando: generate_screen_from_text
   Argumentos: {"deviceType": "DESKTOP", "projectId": "14478153488856050545",
"prompt": "Pantalla de inicio de sesión (login) para escritorio, con TODOS
los textos en español. Fondo claro y una tarjeta blanca centrada... Botón
principal ancho completo en azul oscuro (#1E3A8A)..."}
Acción completada con éxito.
Paso 3: el agente está pensando...
Agente: Pantalla generada. Verifico el resultado.
Acción del Agente -> Ejecutando: list_screens
Acción completada con éxito.
Paso 4: el agente está pensando...
Acción del Agente -> Ejecutando: get_project
Acción completada con éxito.
Paso 5: el agente está pensando...
Acción del Agente -> Ejecutando: list_screens
Acción completada con éxito.
Paso 6: el agente está pensando...
Acción del Agente -> Ejecutando: download_assets
Acción completada con éxito.
Paso 7: el agente está pensando...
Acción del Agente -> Ejecutando: download_file
Acción completada con éxito.
Paso 8: el agente está pensando...
Acción del Agente -> Ejecutando: list_screens
Acción completada con éxito.
Acción del Agente -> Ejecutando: list_design_systems
Acción completada con éxito.
Se alcanzó el límite de 8 pasos. Pidiendo al agente su resumen final...

Agente: Proyecto "Portal de Acceso — Autenticación" creado y pantalla de
inicio de sesión generada para escritorio:
- Tarjeta blanca centrada sobre fondo claro, con título "Iniciar sesión" y
```

```
subtítulo.
- Campos con etiqueta e ícono: Usuario (ícono de persona) y Contraseña (ícono
de candado, con botón de mostrar/ocultar).
- Botón principal ancho en azul oscuro (#1E3A8A) con texto "Iniciar sesión".
- Checkbox "Recordar sesión siempre" (marcado) y enlace "¿Olvidaste tu
contraseña?".
- Pie con "¿No tienes cuenta?" + enlace "Regístrate".
- Se aplicó el sistema de diseño "Executive Slate" (Inter, esquinas
redondeadas, sombras suaves, grid de 8px) con todos los textos en español.

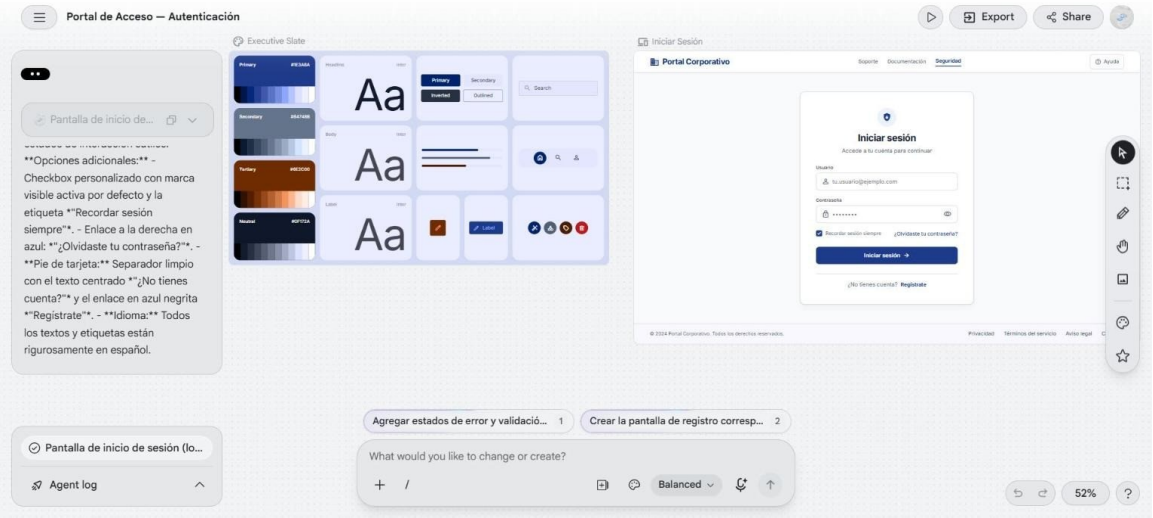
El proyecto es privado y quedó listo en Stitch para revisión y exportación.

Registro guardado en salida_agente.txt
```

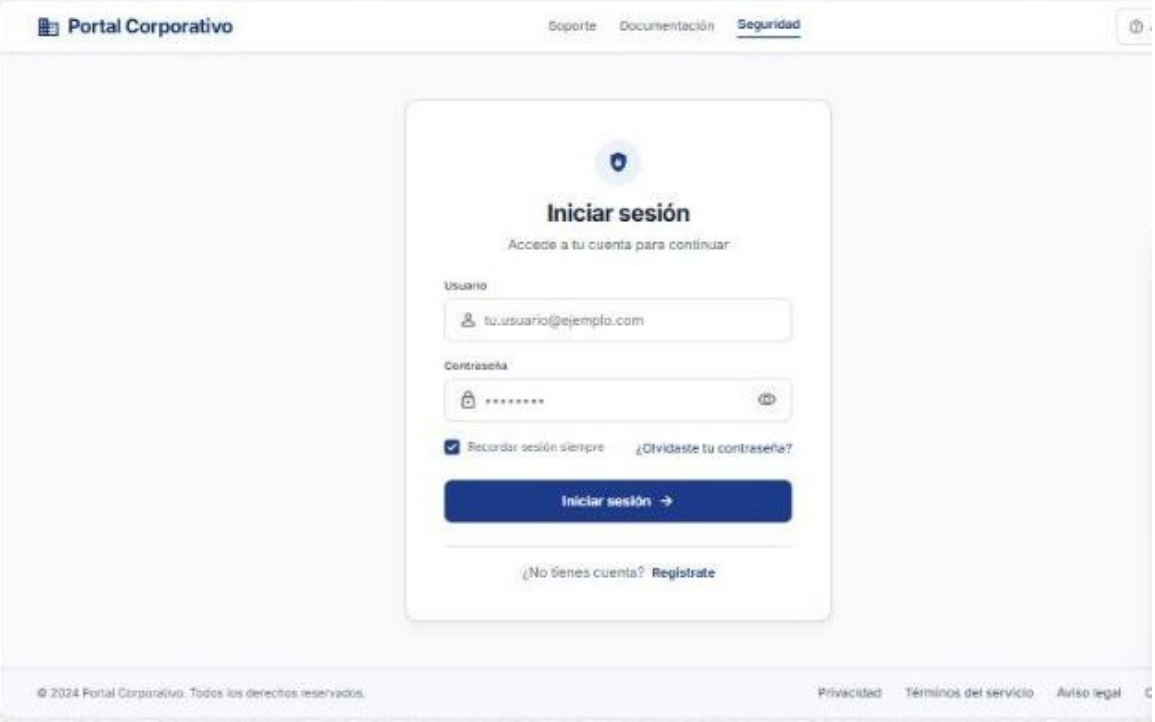
En la ejecución se ve cómo el agente razona y decide por sí mismo:

1. Se conecta a Stitch MCP y descubre **17 herramientas**.
2. Cuando el modelo principal no tiene saldo (402) o no está disponible (403), cambia solo a un modelo gratuito de OpenRouter.
3. **Paso 1:** usa `create_project` y crea el proyecto "Portal de Acceso — Autenticación".
4. **Paso 2:** usa `generate_screen_from_text` con el ID que recibió en el paso anterior y un prompt detallado que él mismo redactó.
5. **Pasos 3 a 8:** verifica su trabajo con `list_screens`, `get_project`, `list_design_systems` y `download_file`.
6. Al llegar al límite de pasos entrega el resumen final.

Resultado en Stitch



Captura 3. Proyecto "Portal de Acceso — Autenticación" creado por el agente en Stitch, con el sistema de diseño "Executive Slate" y la pantalla generada.



Captura 4. Pantalla de inicio de sesión generada por el agente.

La pantalla cumple con todo lo que pide el USER PROMPT:

Lo que pide el USER PROMPT	Resultado en la pantalla
Proyecto nuevo	Proyecto "Portal de Acceso — Autenticación"
Pantalla de inicio de sesión	Pantalla "Iniciar Sesión" para escritorio
Recibe usuario y password	Campos **Usuario** y **Contraseña** con icono
Botón azul oscuro	Botón "Iniciar sesión" azul oscuro (#1E3A8A)
Recordar sesión siempre	Casilla "Recordar sesión siempre"
Olvidé mi contraseña	Enlace "¿Olvidaste tu contraseña?"
Registrarse	"¿No tienes cuenta? Regístrate"

Problemas encontrados y soluciones

Problema	Solución
Al código del manual le faltaba cargar el `.env`, crear el cliente `openai` y declarar `mensajes` y `tareaCompletada`	Se completaron esas partes respetando la estructura del manual
`npx -y @google/stitch-sdk` no levanta ningún servidor: el paquete no trae un comando ejecutable	Se creó `stitch-mcp.js`, un servidor MCP local por stdio con el `StitchProxy` del mismo

Problema	Solución
	paquete oficial
El modelo del manual, `anthropic/claude-3.5-sonnet`, ya fue retirado por Anthropic	Se usó Claude Haiku 4.5 a través de OpenRouter, configurable en `.env`
Error 402 de OpenRouter: el agente pedía hasta 65 536 tokens de respuesta y el saldo no alcanzaba	Se limitó la respuesta con `max_tokens: 2000` y se usó un modelo más económico
Generar una pantalla en Stitch tarda varios minutos y el cliente MCP corta a los 60 segundos	Se dio un tiempo de espera de 10 minutos a `callTool`
Sin saldo en OpenRouter (402) y modelos gratuitos bloqueados para uso directo (403)	Respaldo automático: el agente prueba el siguiente modelo gratuito con herramientas hasta encontrar uno disponible
El modelo gratuito seguía verificando y se acababan los pasos antes del resumen	Al llegar al límite de pasos se le pide un resumen final sin herramientas

Conclusión

La práctica muestra la diferencia entre un chatbot y un agente: el modelo no solo responde, sino que decide qué herramientas usar, en qué orden y con qué datos. En la ejecución, el agente creó primero el proyecto, tomó el ID que le devolvió Stitch y lo usó para generar la pantalla, sin que yo le dijera cómo hacerlo.

MCP es lo que hace posible separar el cerebro de las manos: el mismo bucle funcionaría con otro modelo de OpenRouter o con otro servidor MCP, porque las herramientas se descubren y se traducen automáticamente. También aprendí que el código de ejemplo hay que revisarlo, ya que los paquetes y los modelos cambian con el tiempo.